UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO" — UNESP
COLÉGIO TÉCNICO INDUSTRIAL "PROF. ISAAC PORTAL ROLDÁN" — CTI
HABILITAÇÃO PROFISSIONAL TÉCNICA EM INFORMÁTICA — TURMA INI3A

Arthur Moreira Felicio – Nº 04
Gabriel Camaforte dos Santos – Nº 08
Gabriel Sulzbacher Coimbra de Souza – Nº 09
Heitor Goulart Utyama – Nº 12
Leonardo Godoy dos Anjos – Nº 17
Maria Luiza Fagioli dos Santos Alves – Nº 23

PRESCO

Manual Técnico do Sistema

Manual Técnico: Arquitetura, Modelagem, Implantação e Operação

Sistema & Versão:	Plataforma PResco — Release Oficial v1.5.1
Escopo Documental:	Especificação arquitetural full-stack, API REST, diagramas UML/DER, setup local e deploy em produção
Instituição / Curso:	CTI — UNESP Campus de Bauru (Técnico em Informática — INI3A)

BAURU
2026

ARTHUR MOREIRA FELICIO – Nº 04
GABRIEL CAMAFORTE DOS SANTOS – Nº 08
GABRIEL SULZBACHER COIMBRA DE SOUZA – Nº 09
HEITOR GOULART UTYAMA – Nº 12
LEONARDO GODOY DOS ANJOS – Nº 17
MARIA LUIZA FAGIOLI DOS SANTOS ALVES – Nº 23

PRESCO

Manual Técnico: Arquitetura, Modelagem, Implantação e Operação

Trabalho de Conclusão de Curso (TCC) apresentado ao Colégio Técnico Industrial "Prof. Isaac Portal Roldán" (CTI) da Universidade Estadual Paulista "Júlio de Mesquita Filho" (UNESP) – Campus de Bauru, como requisito parcial obrigatório para obtenção do título de Técnico em Informática (Turma INI3A).

Equipe e Atribuições Técnicas:

Nº	Integrante	Papel / Função Principal
04	Arthur Moreira Felicio	Scrum Master & Desenvolvedor Full-Stack
08	Gabriel Camaforte dos Santos	Tech Lead & Desenvolvedor Full-Stack
09	Gabriel Sulzbacher Coimbra de Souza	Product Owner & Desenvolvedor Full-Stack
12	Heitor Goulart Utyama	Desenvolvedor & Qualidade (QA)
17	Leonardo Godoy dos Anjos	Desenvolvedor & Banco de Dados
23	Maria Luiza Fagioli dos Santos Alves	Designer UI/UX & Desenvolvedora Frontend

BAURU
2026

SUMÁRIO

1 CONTEXTUALIZAÇÃO E MOTIVAÇÃO **4**
1.1 O Problema da Assimetria e Dispersão de Preços **4**
1.2 Objetivos do Sistema PResco **4**
1.3 Público-Alvo e Escopo Operacional **5**
2 TECNOLOGIAS UTILIZADAS E ARQUITETURA DE SOFTWARE **5**
2.1 Matriz de Tecnologias, Linguagens e Frameworks **5**
2.2 Padrão de Arquitetura do Backend (Modular Layered Architecture) **7**
2.3 Padrão de Arquitetura do Frontend Mobile e Web **8**
2.4 Catálogo Completo de Endpoints da API REST **9**
2.5 Mapeamento das Rotas e Telas do Aplicativo Mobile **12**
3 DIAGRAMAS DO SISTEMA **13**
3.1 Diagrama de Classes (UML) **13**
3.2 Diagrama Entidade-Relacionamento / Lógico do Banco de Dados (DER) **14**
3.3 Dicionário de Dados e Índices Geoespaciais/Trigramas **15**
4 GUIA PASSO A PASSO — SETUP DE DESENVOLVIMENTO **16**
4.1 Pré-requisitos de Software e Ferramentas **16**
4.2 Clonagem e Instalação Automatizada (One-Command Setup) **16**
4.3 Configuração Manual Passo a Passo (Variáveis .env, Docker e Banco) **17**
4.4 Inicialização dos Servidores e Modos de Rede **18**
4.5 Execução da Suíte de Testes e Auditoria Local **19**
5 GUIA PASSO A PASSO — SETUP DE PRODUÇÃO E DEPLOY **19**
5.1 Arquitetura de Produção e Higiene Estrita do Servidor **19**
5.2 Montagem e Configuração do Banco de Dados e Cache em Produção **20**
5.3 Configuração do Servidor Web (Apache Reverse Proxy, PM2 e Segurança) **20**
5.4 Procedimento de Build e Distribuição Mobile (APK Android e IPA iOS) **21**
5.5 Procedimento de Build e Deploy da Landing Page Web (GitHub Pages) **22**
5.6 Esteira de CI/CD de 4 Camadas (GitHub Actions) **23**

1. CONTEXTUALIZAÇÃO E MOTIVAÇÃO

1.1. O Problema da Assimetria e Dispersão de Preços

No varejo supermercadista e no comércio local brasileiro, observa-se uma acentuada dispersão de preços para produtos idênticos (identificados pelo mesmo código de barras internacional EAN-13) comercializados em estabelecimentos situados a poucos quilômetros de distância. Esse fenômeno econômico é alimentado pela assimetria de informação: enquanto as redes varejistas dispõem de sistemas automatizados de precificação dinâmica, o consumidor final enfrenta um elevado custo de busca (*search cost*), precisando deslocar-se fisicamente entre múltiplos mercados ou consultar encartes impressos fragmentados e rapidamente desatualizados.

Adicionalmente, mesmo quando o consumidor descobre que determinados itens estão mais baratos em mercados distintos, surge um dilema logístico e financeiro: **o custo do deslocamento veicular (combustível e distância percorrida) compensa a economia obtida na gôndola?** Sem uma ferramenta analítica georreferenciada, o consumidor frequentemente gasta em combustível parte significativa daquilo que economizaria dividindo suas compras em múltiplos supermercados.

1.2. Objetivos do Sistema PResco

O sistema **PResco** foi concebido, arquitetado e implementado como uma plataforma *full-stack* de **monitoramento colaborativo de preços (*crowdsourcing*) baseada em geolocalização espacial e otimização matemática de rotas de compras**. Seus pilares fundamentais são:

1. **Transparência de Preços em Tempo Real via *Crowdsourcing*:** Permitir que consumidores escaneiem códigos de barras EAN-13 diretamente pela câmera do smartphone ou pesquisem itens no catálogo, consultando e registrando cotações reais vinculadas geograficamente aos supermercados da sua região (raio padrão de 15 km processado via **PostGIS**).
2. **Defesa Estatística Anti-Fraude e Curadoria Comunitária:** Garantir a integridade dos dados por meio de validação de limites realistas (`[R$ 0,10, R$ 15.000,00]`), detecção estatística de *outliers* via Desvio Absoluto Mediano ($\text{MAD}_{\text{eff}} = \max(\text{MAD}, 0{,}05 \times \text{mediana})$), isolamento de *Cold Start* (primeira cotação provisória até corroboração dupla em faixa de $\pm 20\%$) e sistema de votação comunitária (*upvotes* e *downvotes*) aliado a um painel administrativo de moderação.
3. **Motor Inteligente de Otimização de Lista de Compras Multilojas (*Travel Cost Optimizer*):** Calcular automaticamente a alocação ideal do carrinho de compras entre supermercados próximos, confrontando o preço unitário dos produtos com a distância real de condução (via **HERE Routing API v8** e cálculo geodésico de Haversine), o consumo médio do veículo (km/L) e o preço do combustível (R$/L), oferecendo três estratégias: *Economia Máxima* (`max_savings`), *Balanceado* (`balanced`) e *Loja Única* (`single_store`).

4. **Engajamento por Gamificação e Reputação:** Incentivar a colaboração contínua sem custo financeiro ao usuário mediante atribuição de pontos de experiência (**XP**), progressão de níveis e cargos (*Roles*), desbloqueio de conquistas (*Badges* de 0 a 10.000 XP) e loja de personalização cosmética de perfil (banners, molduras de avatar, insígnias de nível e títulos).

1.3. Público-Alvo e Escopo Operacional

O público-alvo do **PResco** abrange:

- **Consumidores Domésticos e Famílias:** Cidadãos que buscam reduzir o orçamento mensal de supermercado comparando preços na sua cidade (com foco inicial validado em Bauru/SP e suporte dinâmico a todo o território nacional via descoberta automática *OpenStreetMap*).
- **Comunidade Colaborativa de Compradores:** Usuários engajados que atualizam cotações no ponto de venda e auditam preços de outros colaboradores.
- **Moderadores e Administradores da Plataforma:** Responsáveis pela curadoria de ocorrências suspeitas (*outliers*), gestão do catálogo de produtos e manutenção dos estabelecimentos comerciais.

2. TECNOLOGIAS UTILIZADAS E ARQUITETURA DE SOFTWARE

2.1. Matriz de Tecnologias, Linguagens e Frameworks

Todas as versões listadas na Tabela 1 foram extraídas diretamente dos manifestos oficiais do repositório (`package.json` , `src/backend/package.json` , `src/frontend/package.json` , `src/landing/package.json` e `docker-compose.yml`).

Tabela 1 — Stack Tecnológica Completa do Sistema PResco (v1.5.1)

Camada / Domínio	Tecnologia / Biblioteca	Versão Exata	Finalidade Técnica no Projeto
Monorepo & Build	Node.js (Engine)	`>= 20.11.0`	Ambiente de execução JavaScript/TypeScript servidor e tooling
Monorepo & Build	NPM Workspaces	`10.9.2`	Gerenciamento unificado de pacotes (`src/backend` , `src/frontend`)
Monorepo & Build	Turborepo (`turbo`)	`^2.5.4`	Orquestração paralela de pipelines (`dev` , `build` , `db:migrate` , `db:seed`)
Linguagem Base	TypeScript	`^6.0.3`	Tipagem estática estrita em 100% do backend, frontend, scripts e landing page
Backend Runtime	`tsx`	`^4.21.0`	Execução direta e *watch mode* de código TypeScript ESM sem etapa de transpilação pesada
Backend Framework	Express.js (`express`)	`^5.2.1`	Servidor HTTP REST, roteamento modular e cadeia de *middlewares*

Camada / Domínio	Tecnologia / Biblioteca	Versão Exata	Finalidade Técnica no Projeto
ORM & Migrações	Drizzle ORM (`drizzle-orm`)	`^0.45.2`	Mapeamento objeto-relacional *type-safe* e construtor SQL parametrizado
ORM CLI	Drizzle Kit (`drizzle-kit`)	`^0.31.10`	Geração e versionamento de migrações SQL (`0000` a `0005`)
Banco de Dados	PostgreSQL + PostGIS	`17-3.5`	SGBD relacional com extensão espacial geodésica (`geography`, índices `GiST`) e trigramas (`GIN`)
Driver DB	`pg` (node-postgres)	`^8.21.0`	Pool de conexões assíncronas com PostgreSQL
Cache & Sessões	Redis (`redis`)	`7-alpine` / `^6.0.0`	Armazenamento de *Refresh Tokens*, *Blacklist* de JTI, *Rate Limiting* e *Cache-Aside* HTTP
Segurança & Auth	`jsonwebtoken` + `bcrypt`	`^9.0.3` / `^6.0.0`	Assinatura de Access Tokens JWT (15 min) e hashing criptográfico de senhas (custo 10)
Validação & Mídia	`zod` + `sharp` + `compression`	`^4.4.3` / `^0.35.4`	Validação de esquemas, transcodificação de imagens para WebP/AVIF e compressão Brotli/Gzip
Frontend Mobile	React Native + React	`0.86.3` / `19.2.3`	Renderização nativa multiplataforma (Android e iOS) com Nova Arquitetura (Fabric/JSI)
Ecossistema Mobile	Expo SDK (`expo`)	`57.0.20`	Abstração de APIs nativas, *prebuild* e empacotamento Metro Bundler
Roteamento Mobile	Expo Router (`expo-router`)	`~57.0.19`	Roteamento declarativo baseado em arquivos (`src/frontend/app/`) e *deep linking*
Animações & Gestos	React Native Reanimated + Gesture Handler	`4.5.1` / `~2.32.0`	Animações a 60/120 FPS na *UI Thread* e navegação por deslize horizontal 1:1 (*SwipeTabNavigator*)
Geolocalização & Mapas	`react-native-maps` + `expo-location`	`1.27.2` / `~57.0.16`	Mapa interativo nativo de supermercados, leitura de GPS em tempo real e cálculo de rotas
Câmera & Óptica	`expo-camera` + `expo-image`	`~57.0.4`	Decodificação acelerada de códigos de barras 1D (`ean13`, `ean8`, `upc_a`, `code128`) e cache de imagem
Persistência Local	`@react-native-async-storage/async-storage`	`2.2.0`	Persistência local de tokens JWT, carrinho *offline-first*, preferências de veículo e tema

Camada / Domínio	Tecnologia / Biblioteca	Versão Exata	Finalidade Técnica no Projeto
Landing Page Web	React Native Web (`react-native-web`)	`^0.21.2`	Portal institucional e página de download com Canvas 2D procedural e QR Code ISO/IEC 18004
Qualidade & Testes	Node Test Runner + Grafana k6 + Maestro	Nativo / CLI	Testes unitários, de integração, condições de corrida, carga/estresse (k6) e E2E mobile (Maestro)

Fonte: Elaborado pelos autores (2026).

2.2. Padrão de Arquitetura do Backend (Modular Layered Architecture)

O backend do **PResco** (`src/backend/src/`) adota uma **Arquitetura em Camadas Estrita orientada a Domínios (Modular DDD / Layered Architecture)**. O fluxo de controle de toda requisição HTTP obedece invariavelmente à cadeia unidirecional:

Routes ⟶ Controller ⟶ Service ⟶ Repository ⟶ PostgreSQL (PostGIS) / Redis

1. **Camada de Rotas (`modules/[modulo]/[modulo].routes.ts`):**
   - Cria a instância `Router()` do Express 5.
   - Acopla os *middlewares* transversais: autenticação (`requireAuth`, `optionalAuth`, `requireTwoFactor`), autorização por nível de autoridade (`requireAdmin`), limitação de taxa (`authRateLimiter`, `searchRateLimiter`) e cache HTTP (`cacheResponse`).
   - Encaminha a chamada ao método correspondente do *Controller Singleton*. Não contém nenhuma lógica de negócio ou query SQL.
2. **Camada de Controladores (`modules/[modulo]/[modulo].controller.ts`):**
   - Implementada como classes *Singleton* (ex.: `authController`, `productController`, `ocurrencyController`, `marketController`, `cartController`, `customizationController`).
   - Recebe `(req: Api.Request, res: Response, next: NextFunction)` dentro de blocos `try/catch`, encaminhando exceções para o `errorHandler` global via `next(e)`.
   - Valida os parâmetros de entrada (`req.body`, `req.params`, `req.query`), acumulando inconsistências em vetores `{ field, message }[]` e disparando `ValidationError` (HTTP 422) quando aplicável.
   - Padroniza todas as respostas bem-sucedidas com o envelope `success(data)` (`{ success: true, code: 200, data }`).
3. **Camada de Serviços (`modules/[modulo]/[modulo].service.ts`):**
   - Concentra 100% das regras de negócio da aplicação: criptografia de senhas (`bcrypt`), emissão e rotação de tokens JWT/Refresh, motor estatístico de validação de preços

(MAD_{eff} e *Cold Start*), cálculo de recompensas de gamificação (**+15 XP** por cotação, **+5 XP** por voto, **+25 XP** por novo produto), descoberta automática de mercados via *OpenStreetMap* (`OsmMarketDiscovery`) e algoritmo de otimização de rotas multilojas (`cartService`).
    - É totalmente agnóstica a protocolo HTTP e lança apenas subclasses tipadas de `AppError` (`UnauthorizedError` , `ForbiddenError` , `NotFoundError` , `ConflictError` , `ValidationError` , `TooManyRequestsError`).
4. **Camada de Repositórios (`shared/database/repositories/[entidade].repository.ts`):**
    - **Único local autorizado** a invocar o **Drizzle ORM** (`db.select` , `db.insert` , `db.update` , `db.delete`), funções geoespaciais **PostGIS** (`ST_DWithin` , `ST_Distance` , `ST_AsGeoJson`) ou comandos **Redis** (`AuthRepository`).

2.3. Padrão de Arquitetura do Frontend Mobile e Web

O aplicativo móvel (`src/frontend/`) foi construído sobre **React Native 0.86.3** e **Expo SDK 57**, estruturado em três camadas desacopladas:

1. **Roteamento Baseado em Arquivos (`src/frontend/app/`):** Gerenciado pelo `expo-router` , com layout raiz (`_layout.tsx`) que provê os contextos globais e encapsula as abas principais no navegador gestual `SwipeTabNavigator.tsx` (rastreamento de toque 1:1 com física de mola via *Reanimated 4*).
2. **Provedores de Estado e Contexto (`src/frontend/content/`):**
    - `authContext.tsx` : Gerencia a sessão ativa do usuário (`user` , `profile` , `isAdmin` , `isTwoFactorVerified` , `refreshProfile`).
    - `i18nContext.tsx` : Motor de internacionalização tipado com suporte instantâneo a **7 idiomas** (`pt-BR` , `en-US` , `es-ES` , `de-DE` , `ru-RU` , `zh-CN` , `ja-JP`).
    - `theme/index.ts` & `themeContent.tsx` : Sistema de Design baseado em tokens primitivos e semânticos (`DESIGN.md`), com suporte aos modos **Claro (Light)**, **Escuro (Dark)**, **AMOLED Puro (`#000000`)** e extração dinâmica de paletas **Material You (Monet)**.
3. **Camada de Serviços de Domínio (`src/frontend/services/`):**
    - Centralizada sobre o cliente HTTP resiliente `apiRequest<T>` (`services/api.ts`), que injeta automaticamente o cabeçalho `Authorization: Bearer <token>` , gerencia *timeout* de 15 segundos via `AbortController` e executa renovação transparente de sessão (*Token Refresh*) ao detectar HTTP 401.
    - Serviços especializados: `auth.ts` , `productService.ts` , `ocurrencyService.ts` , `marketService.ts` , `cartService.ts` e `customizationService.ts` .

2.4. Catálogo Completo de Endpoints da API REST

A Tabela 2 documenta todos os endpoints reais expostos pelo backend em `src/backend/src/app.ts` e seus respectivos módulos de rota.

Tabela 2 — Mapeamento Completo dos Endpoints da API REST do PResco

Módulo	Método	Rota / Endpoint	Autenticação / Middleware	Descrição Funcional
Infra & Saúde	`GET`	`/`, `/index.html`, `/index.php`	Público	Retorna metadados da API, versão ativa e links de saúde
Infra & Saúde	`GET`	`/health`, `/ping`, `/products/barcode/ping`	Público	Verifica conectividade com PostgreSQL/PostGIS e Redis
Autenticação	`POST`	`/auth/register`	Público (`authRateLimiter`)	Cadastra novo usuário, atribui Role inicial e Badge #1 (*Pioneiro*)
Autenticação	`POST`	`/auth/login`	Público (`authRateLimiter`)	Autentica credenciais e retorna `accessToken` (JWT) + `refreshToken`
Autenticação	`POST`	`/auth/refresh`	Público	Rotaciona atomicamente o `refreshToken` no Redis e emite novo JWT
Autenticação	`GET`	`/auth/me`	`requireAuth`	Retorna perfil completo, XP, nível, conquistas, estatísticas e cosméticos
Autenticação	`POST`	`/auth/logout`	`requireAuth`	Revoga `refreshToken` e insere `jti` do JWT na *blacklist* do Redis
Autenticação	`POST`	`/auth/2fa/send`	`requireAuth` (`authRateLimiter`)	Dispara código OTP de 6 dígitos por e-mail para verificação em 2 etapas
Autenticação	`POST`	`/auth/2fa/verify`	`requireAuth` (`authRateLimiter`)	Valida o código OTP e ativa a flag `twoFactorVerified` do usuário
Autenticação	`PATCH`	`/auth/password`	`requireAuth` (`authRateLimiter`)	Altera a senha do usuário mediante validação da senha atual
Autenticação	`DELETE`	`/auth/account`	`requireAuth`	Exclui permanentemente a conta (conformidade LGPD) e revoga tokens
Produtos	`GET`	`/products`	Público (`cacheResponse` 60s)	Lista produtos com filtro por busca textual (trigrama), categoria e raio GPS

Módulo	Método	Rota / Endpoint	Autenticação / Middleware	Descrição Funcional
Produtos	`GET`	`/products/categories`	Público (`cacheResponse` 1h)	Lista as categorias normalizadas presentes no catálogo
Produtos	`GET`	`/products/categories/details`	Público (`cacheResponse` 1h)	Retorna as 15 categorias canônicas com ícones, emojis e metadados
Produtos	`GET`	`/products/barcode/:ean`	Público (`searchRateLimiter`)	Busca produto por código EAN-13 (banco local + fallback OpenFoodFacts/Cosmos)
Produtos	`GET`	`/products/:id`	Público (`cacheResponse` 120s)	Obtém detalhes do produto, estatísticas (mínimo, médio, máximo) e mercado mais próximo
Produtos	`GET`	`/products/:id/history`	Público (`cacheResponse` 120s)	Retorna série histórica de evolução de preços para renderização de gráfico
Produtos	`POST`	`/products` ou `/products/custom`	`requireTwoFactor`	Cadastra novo produto no catálogo e concede **+25 XP** ao autor
Produtos	`POST`	`/products/:id/report`	`requireTwoFactor`	Registra denúncia comunitária de problema em cadastro de produto
Produtos	`PUT` / `PATCH`	`/products/:id`	`requireAdmin`	Atualiza dados cadastrais de um produto (exclusivo Administrador)
Produtos	`DELETE`	`/products/:id`	`requireAdmin`	Remove um produto do catálogo (exclusivo Administrador)
Cotações	`GET`	`/ocurrency/product/:productId`	`optionalAuth`	Lista ocorrências de preço ativas para o produto (ordenadas por proximidade/preço)
Cotações	`POST`	`/ocurrency`	`requireTwoFactor`	Registra nova cotação de preço em um mercado (**+15 XP**, sujeito a validação MAD)
Cotações	`POST`	`/ocurrency/:id/vote`	`requireTwoFactor`	Registra ou alterna voto positivo/negativo na cotação (**+5 XP**, bloqueia auto-voto)
Cotações	`PUT`	`/ocurrency/:id`	`requireTwoFactor`	Edita valor de ocorrência e reexecuta verificação estatística de *outliers*

Módulo	Método	Rota / Endpoint	Autenticação / Middleware	Descrição Funcional
Cotações	`DELETE`	`/ocurrency/:id`	`requireTwoFactor`	Remove uma ocorrência de preço (autor ou administrador)
Moderação	`GET`	`/ocurrency/admin/pending`	`requireAuth`, `requireAdmin`	Lista ocorrências suspensas/pendentes na fila de moderação administrativa
Moderação	`POST`	`/ocurrency/:id/approve`	`requireAuth`, `requireAdmin`	Aprova cotação retida, publica no catálogo e libera XP pendente de forma idempotente
Moderação	`POST`	`/ocurrency/:id/reject`	`requireAuth`, `requireAdmin`	Rejeita cotação fraudulenta/incorreta e encerra a pendência
Mercados	`GET`	`/markets`	Público	Lista supermercados num raio espacial PostGIS (15 km) + descoberta automática OSM
Mercados	`GET`	`/markets/:id`	Público	Consulta dados e coordenadas geográficas (`ST_AsGeoJson`) de um mercado específico
Mercados	`POST`	`/markets`	`requireAuth`	Cadastra novo estabelecimento comercial (valida envelope territorial brasileiro)
Mercados	`PUT`	`/markets/:id`	`requireAuth`, `requireAdmin`	Atualiza nome ou localização geográfica de um mercado
Mercados	`DELETE`	`/markets/:id`	`requireAuth`, `requireAdmin`	Remove estabelecimento comercial
Carrinho	`GET`	`/cart`	`requireAuth`	Recupera itens salvos no carrinho persistente do usuário autenticado
Carrinho	`POST`	`/cart/items`	`requireAuth`	Adiciona produto ao carrinho ou incrementa sua quantidade
Carrinho	`PUT`	`/cart/items/:productId`	`requireAuth`	Atualiza a quantidade exata de um item no carrinho
Carrinho	`DELETE`	`/cart/items/:productId`	`requireAuth`	Remove um produto específico do carrinho
Carrinho	`DELETE`	`/cart`	`requireAuth`	Esvazia integralmente o carrinho do usuário

Módulo	Método	Rota / Endpoint	Autenticação / Middleware	Descrição Funcional
Otimizador	`POST`	`/cart/optimize`	`optionalAuth`	Executa o motor de otimização multilojas com trade-off de combustível e distância
Customização	`GET`	`/customizations/shop`	`requireAuth`	Lista catálogo da loja de customização de perfil e itens já desbloqueados
Customização	`POST`	`/customizations/buy/:itemId`	`requireAuth`	Desbloqueia item cosmético (por marco de nível/XP, sem debitar XP total)
Customização	`POST`	`/customizations/equip/:itemId`	`requireAuth`	Equipa banner, moldura de avatar, moldura de nível ou título no perfil
Customização	`POST`	`/customizations/unequip/:category`	`requireAuth`	Desequipa o item cosmético ativo de uma categoria
Mídia CDN	`GET`	`/images/optimize`	Público	Proxy otimizador que redimensiona e converte imagens externas para WebP/AVIF via Sharp

Fonte: Elaborado pelos autores (2026).

2.5. Mapeamento das Rotas e Telas do Aplicativo Mobile

A Tabela 3 apresenta todas as telas implementadas em `src/frontend/app/` com suas responsabilidades funcionais.

Tabela 3 — Mapeamento das Telas do Aplicativo Mobile (src/frontend/app/)

Arquivo de Rota	Nome da Tela	Principais Funcionalidades
`_layout.tsx`	Layout Raiz & Providers	Inicializa contextos de Tema, i18n, Autenticação, Gestos e limpeza de memória em *background*
`index.tsx`	Home (Início)	Carrossel de destaques, atalhos rápidos, radar de ofertas próximas (15 km) e supermercados da região
`search.tsx`	Catálogo & Busca	Busca em tempo real com *debounce*, filtro pelas 15 categorias canônicas, atalho de scanner e paginação infinita
`scannerProduct.tsx`	Scanner EAN-13	Leitor óptico de código de barras via câmera nativa com mira animada, lanterna e busca concorrente
`scannerConfirmation.tsx`	Confirmação de Leitura	Confirmação visual do produto escaneado ("Este é o produto escaneado?") em tela única sem rolagem

Arquivo de Rota	Nome da Tela	Principais Funcionalidades
`manualEanSearch.tsx`	Digitação Manual EAN	Entrada manual de dígitos do código de barras quando o rótulo físico está danificado
`customRegisterProduct.tsx`	Cadastro de Produto	Formulário para inclusão colaborativa de novos produtos no catálogo (**+25 XP**)
`registerProduct.tsx`	Registro de Preço	Submissão de cotação (**+15 XP**) com pré-seleção automática do mercado mais próximo e máscara de centavos
`productDetails.tsx`	Detalhes do Produto	Estatísticas de preço, gráfico de histórico temporal, ranking de mercados, votação (**+5 XP**) e adição ao carrinho
`cart.tsx`	Lista & Otimizador	Carrinho inteligente com agrupamento por supermercado, seletor de estratégia, realocação de itens e rota GPS
`map.native.tsx` / `map.web.tsx`	Mapa de Mercados	Mapa georreferenciado com pinos por categoria (Supermercados, Mercados Locais, Hortifrutis) e rota no Google Maps
`profile.tsx`	Perfil Gamificado	Nível, barra de XP, título, conquistas (15 medalhas), *heatmap* de contribuições e Loja de Customização com *preview*
`settings.tsx`	Configurações	Temas (Light/Dark/AMOLED), cores Monet, idioma (7 opções), backup/importação, limpeza de cache, senha e LGPD
`adminModeration.tsx`	Painel de Moderação	Fila exclusiva para Administradores aprovarem ou rejeitarem cotações retidas por desvio estatístico
`help.tsx`	Central de Ajuda	18 perguntas frequentes (FAQ) organizadas em abas interativas com busca instantânea e guia de XP
`about.tsx`	Sobre o PResco	Missão institucional do projeto, stack tecnológica, versão SemVer e links de código aberto
`login.tsx` / `registerUser.tsx`	Autenticação	Telas de entrada e criação de conta com validação de campos e *1-Tap Quick Login* em modo de desenvolvimento

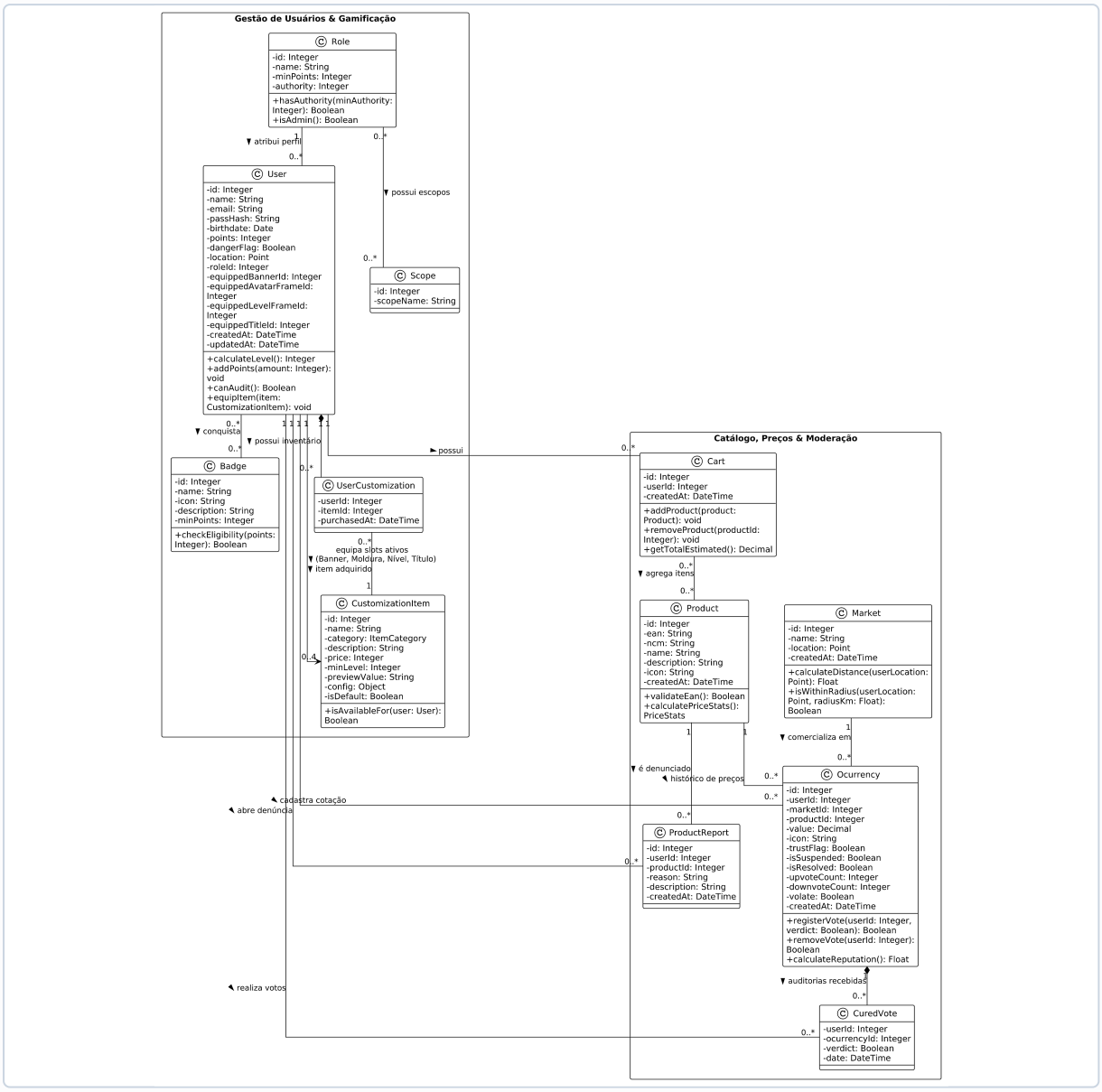
Fonte: Elaborado pelos autores (2026).

3. DIAGRAMAS DO SISTEMA

3.1. Diagrama de Classes (UML)

O Diagrama de Classes a seguir representa fielmente a estrutura arquitetural de classes *Singleton* do backend (`Controllers`, `Services` e `Repositories`) e o encapsulamento em camadas adotado pelo **PResco**. A imagem oficial em alta resolução encontra-se renderizada abaixo:

Figura 1 — Diagrama de Classes UML da Arquitetura do Sistema PResco

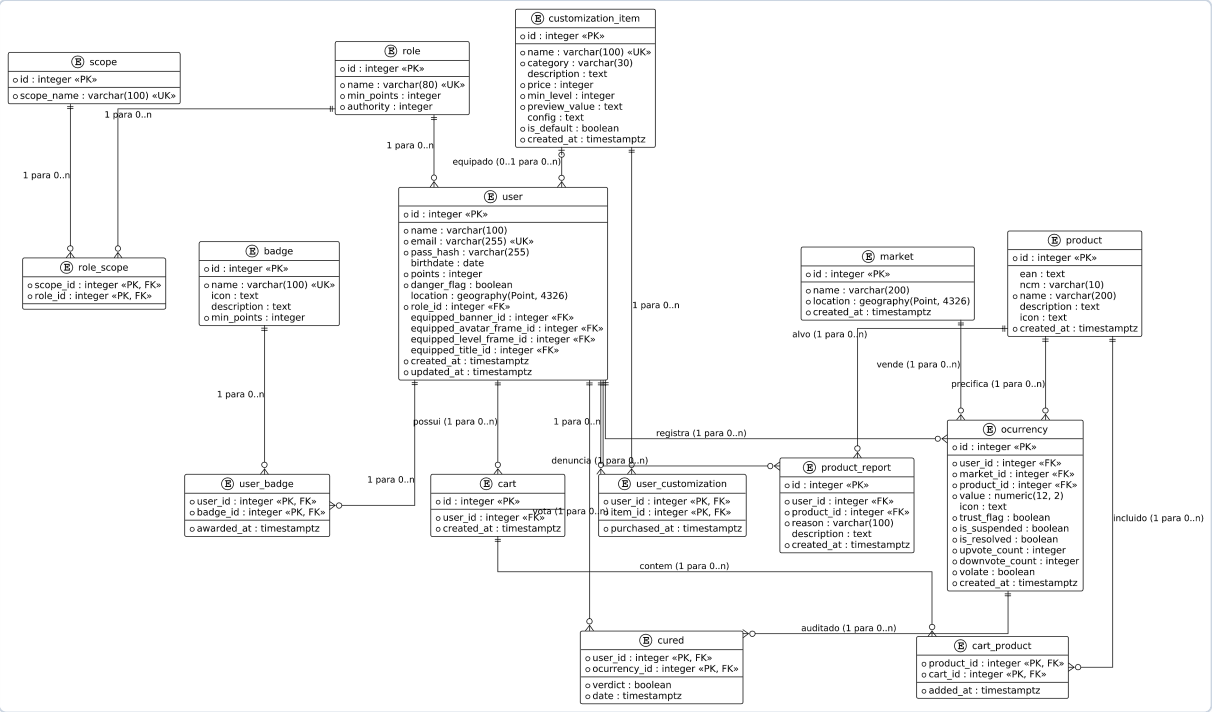


Fonte: Elaborado pelos autores (2026).

3.2. Diagrama Entidade-Relacionamento / Lógico do Banco de Dados (DER)

O modelo relacional do **PResco** é composto por 14 tabelas normalizadas implementadas via **Drizzle ORM** sobre **PostgreSQL 17** com extensão espacial **PostGIS 3.5**. O diagrama visual completo em alta definição é apresentado a seguir:

Figura 2 — Diagrama Entidade-Relacionamento (DER) do Banco de Dados PostgreSQL/PostGIS



Fonte: Elaborado pelos autores (2026).

3.3. Dicionário de Dados e Índices Geoespaciais/Trigramas

O banco de dados do **PResco** utiliza recursos avançados do PostgreSQL 17 para garantir respostas em menos de 50 ms mesmo sob carga concorrente:

1. **Índices Geoespaciais `GiST` (`PostGIS`):**
   - `idx_market_location` na tabela `market(location)` e `idx_user_location` na tabela `user(location)` utilizando a classe de operadores `gist_geography_ops`.
   - Viabilizam consultas de raio geodésico exato em metros sobre o elipsoide terrestre WGS-84 (`SRID=4326`) via `ST_DWithin(market.location, ST_SetSRID(ST_MakePoint(lng, lat), 4326)::geography, 15000)` e ordenação por proximidade via `ST_Distance`.
2. **Índice Invertido de Trigramas `GIN` (`pg_trgm`):**
   - `idx_product_name_trgm` na tabela `product(name)` com `gin_trgm_ops`, permitindo buscas textuais tolerantes a variações parciais (`ILIKE` e similaridade de trigramas) sem *Full Table Scan*.
3. **Índices Parciais de Alta Seletividade:**
   - `idx_product_ean_partial`: Indexa exclusivamente produtos que possuem código de barras (`WHERE ean IS NOT NULL`).
   - `idx_ocurrency_unresolved_partial`: Indexa apenas ocorrências pendentes de moderação (`WHERE is_resolved = false`), acelerando o carregamento do Painel de

Moderação Administrativo.

4. GUIA PASSO A PASSO — SETUP DE DESENVOLVIMENTO

Esta seção descreve o procedimento completo e reproduzível para configurar, executar e testar o ecossistema **PResco** em ambiente de desenvolvimento local (Windows, Linux ou macOS).

4.1. Pré-requisitos de Software e Ferramentas

Antes de iniciar, certifique-se de que a estação de trabalho possui os seguintes softwares instalados:

- **Git:** `>= 2.40` (para controle de versão e clonagem do repositório).
- **Node.js:** `>= 20.11.0` (recomendado Node.js 22 LTS) e **NPM:** `>= 10.9.2`.
- **Docker Engine & Docker Compose:** Docker Desktop (Windows/macOS) ou Docker CE + Compose Plugin (Linux) para execução isolada dos containers `postgis/postgis:17-3.5` e `redis:7-alpine`.
- **Ambiente Mobile (Opcional para Emulação Local):**
  - **Dispositivo Físico (Recomendado para teste rápido):** Aplicativo **Expo Go** instalado no smartphone Android ou iOS, ou APK de desenvolvimento compilado (`devFlavor`).
  - **Emulador Android Local: JDK 17** (Amazon Corretto ou Eclipse Temurin 17), **Android Studio** com Android SDK Build-Tools 36, NDK 27.1 e uma imagem AVD `x86_64` (API 31 ou superior).

4.2. Clonagem e Instalação Automatizada (One-Command Setup)

O repositório conta com orquestradores automatizados multiplataforma (`setup.sh`, `setup.ps1`, `setup.bat` e `scripts/bootstrap.ts`) que verificam dependências, instalam os pacotes dos *workspaces*, geram os arquivos `.env`, sobem os containers Docker, aplicam as migrações Drizzle ORM e populam o banco de dados com o *seed* inicial.

Passo 1: Clonar o Repositório Oficial

```
git clone https://github.com/criperrr/INI3A-EQ3.git
cd INI3A-EQ3
```

Passo 2: Executar o Script de Setup Automatizado

- **No Linux ou macOS:**

  ```
  chmod +x setup.sh start_project.sh
  ./setup.sh
  ```

- **No Windows (PowerShell):**

  ```
  powershell -ExecutionPolicy Bypass -File .\setup.ps1
  ```

(Alternativamente, dê um duplo clique sobre o arquivo `setup.bat` no Windows Explorer).

4.3. Configuração Manual Passo a Passo (Variáveis .env, Docker e Banco)

Caso o desenvolvedor deseje realizar a configuração etapa por etapa manualmente, siga os comandos reais abaixo a partir da raiz do projeto:

Passo 1: Instalar Dependências do Monorepo (NPM Workspaces)

```
npm install
```

Passo 2: Configurar os Arquivos de Variáveis de Ambiente (.env)

Você pode gerar automaticamente os arquivos `.env` para desenvolvimento local executando:

```
npm run env:local
```

Ou criar manualmente os dois arquivos abaixo:

1. **Arquivo `src/backend/.env` (Backend API):**

```
DATABASE_URL=postgres://postgres:postgres@localhost:5433/presco_db
REDIS_URL=redis://localhost:6380
SERVER_PORT=3333
SERVER_HOST=0.0.0.0
JWT_SECRET=presco-dev-secret-key-2026-unesp-cti
NODE_ENV=development
# Opcionais (APIs externas de rota geodésica e envio de e-mail 2FA):
HERE_API_KEY=
RESEND_API_KEY=
```

2. **Arquivo `src/frontend/.env` (Frontend Mobile Expo):**

```
# Substitua pelo IP da sua máquina na rede Wi-Fi local (ou use npm run
env:local)
EXPO_PUBLIC_API_URL=http://192.168.1.100:3333
```

Passo 3: Subir a Infraestrutura de Banco de Dados e Cache via Docker Compose

O arquivo `docker-compose.yml` provisiona o PostgreSQL 17 com extensão PostGIS 3.5 (mapeado na porta `5433` do host para evitar colisão com instâncias locais na `5432`) e o Redis 7 Alpine (mapeado na porta `6380` do host):

```
npm run db:up
```

Para verificar se os containers `presco_postgres` e `presco_redis` estão com status `healthy`:

```
docker compose ps
```

Passo 4: Executar as Migrações Drizzle ORM e o Seeder Inicial

Com os containers ativos, aplique o esquema relacional (`0000` a `0005`, incluindo extensões `postgis` e `pg_trgm`, tabelas, constraints e índices) e execute o *seed* idempotente (criação das 5 *Roles*, 15 *Badges*, itens da loja de customização, supermercados reais de Bauru/SP e usuário administrador `admin@admin.org` / senha `admin`):

```
npm run db:migrate
npm run db:seed
npm run db:check
```

4.4. Inicialização dos Servidores e Modos de Rede

O **PResco** oferece um lançador inteligente (`scripts/dev_launcher.ts`, acionado por `./start_project.sh` ou `.\start_project.ps1`) que detecta automaticamente o IP da interface de rede e sincroniza o `src/frontend/.env`:

1. **Modo Rede Local Wi-Fi / LAN (Latência Zero 0–5 ms — Recomendado):** Ideal quando o computador e o celular estão na mesma rede doméstica ou roteador/USB Tethering:

   ```
   npm run dev:lan
   ```

2. **Modo Rede Corporativa / Campus com Isolamento AP (Túnel Cloud):** Ideal para redes universitárias/escolares (como o Wi-Fi do campus CTI/Unesp) que bloqueiam comunicação *peer-to-peer* entre dispositivos clientes:

   ```
   npm run dev:corp
   ```

3. **Modo Emulador Local (`localhost`):** Quando o aplicativo roda no emulador Android ou navegador na própria máquina:

   ```
   npm run dev:localhost
   ```

Tabela 4 — Portas Utilizadas no Ambiente de Desenvolvimento Local

Serviço / Componente	Porta Local	URL / Endpoint de Acesso Local
Backend API REST (Express 5)	`3333`	`http://localhost:3333` (Healthcheck: `http://localhost:3333/health`)
Metro Bundler (Expo Frontend)	`8081`	`http://localhost:8081` (QR Code no terminal para Expo Go / Dev Client)
PostgreSQL 17 + PostGIS 3.5 (Docker)	`5433`	`postgres://postgres:postgres@localhost:5433/presco_db`
Redis 7 Alpine (Docker)	`6380`	`redis://localhost:6380`

Fonte: Elaborado pelos autores (2026).

4.5. Execução da Suíte de Testes e Auditoria Local

Para validar a integridade do sistema durante o desenvolvimento, execute os seguintes comandos a partir da raiz:

```
# 1. Verificação estática de tipos TypeScript (Backend + Frontend)
npm run typecheck

# 2. Linter de código do Frontend (ESLint + regras React 19 / Reanimated)
npm run lint

# 3. Suíte completa de testes unitários, de integração e de concorrência
(68+ testes)
npm test

# 4. Diagnóstico autônomo de conectividade (Banco, Redis, Backend e Rede)
npm run dev:check

# 5. Testes de carga e performance com Grafana k6 (requer k6 instalado)
npm run test:k6

# 6. Testes End-to-End (E2E) Mobile no emulador Android com Maestro
npm run test:maestro:quick
```

5. GUIA PASSO A PASSO — SETUP DE PRODUÇÃO E DEPLOY

Esta seção documenta a arquitetura de implantação em produção homologada no servidor institucional do **Colégio Técnico Industrial (CTI — UNESP Bauru)** e nos pipelines de distribuição contínua do **GitHub Releases** e **GitHub Pages**.

5.1. Arquitetura de Produção e Higiene Estrita do Servidor

A infraestrutura de produção do **PResco** é distribuída em três ambientes especializados e interconectados por HTTPS:

1. **Servidor de Produção da API Backend (`projetoscti.com.br`):**
   - Hospedado no servidor Linux institucional do CTI no diretório `/var/www/equipes/26-presco` (acesso SSH via porta `4026`).
   - Atende publicamente sob certificado SSL/TLS no endereço `https://eq.projetoscti.com.br/26-presco` (e espelho individual `https://ra.projetoscti.com.br/2457045`).
   - **Política de Higiene Estrita do Servidor:** O script `deploy/deploy.sh` expurga automaticamente quaisquer pastas de desenvolvimento (`.agents/`, `docs/`, `gestao/`, `sprints/`, `scripts/`, `tests/`, `src/frontend/`), mantendo no servidor remoto **exclusivamente** os artefatos de execução do backend (`src/backend/`, `deploy/deploy.sh`, `ecosystem.config.cjs`, `.htaccess`, `logs/` e um único `README.md`).
2. **Pacote Mobile Nativo (Android APK / iOS IPA):**

- Compilado com a variável `EXPO_PUBLIC_API_URL=https://eq.projetoscti.com.br/26-presco` e distribuído oficialmente via **GitHub Releases** (`app-release.apk`).

3. **Portal Institucional e Landing Page Web:**
   - Compilada estaticamente via Metro Web Bundler e servida globalmente via CDN no **GitHub Pages** (`https://criperrr.github.io/INI3A-EQ3/`).

5.2. Montagem e Configuração do Banco de Dados e Cache em Produção

No ambiente de produção, o banco de dados **PostgreSQL** opera na porta `54432` com as extensões `postgis` e `pg_trgm` habilitadas, acompanhado por instância **Redis** (ou Upstash Redis com *fallback* automático em memória `InMemoryStore` caso a conexão oscile).

Durante cada ciclo de deploy (`deploy/deploy.sh`), o próprio orquestrador executa de maneira idempotente e segura:

```
cd /var/www/equipes/26-presco/src/backend

# 1. Aplica todas as migrações SQL pendentes (0000 a 0005)
npm run db:migrate

# 2. Executa o seeder idempotente (garante roles, badges e itens cosméticos
sem sobrescrever senhas ou dados reais)
npm run db:seed

# 3. Executa a verificação e realocação geodésica de mercados
npm run db:reallocate
```

As variáveis de ambiente de produção nunca são versionadas no Git; elas são injetadas com permissão restrita `chmod 600` em `/var/www/equipes/26-presco/src/backend/.env` pelo orquestrador de deploy:

```
DATABASE_URL=postgres://<usuario>:<senha>@localhost:54432/26-presco
REDIS_URL=rediss://<usuario>:<senha>@<endpoint-redis>:6379
SERVER_PORT=3333
SERVER_HOST=0.0.0.0
JWT_SECRET=<chave-secreta-criptografica-256-bits>
NODE_ENV=production
LOG_DIR=/var/www/equipes/26-presco/logs
HERE_API_KEY=<chave-here-routing-v8>
RESEND_API_KEY=<chave-resend-2fa>
RESEND_FROM=onboarding@resend.dev
ALLOWED_ORIGINS=https://criperrr.github.io,https://eq.projetoscti.com.br
```

5.3. Configuração do Servidor Web (Apache Reverse Proxy, PM2 e Segurança)

1. Proxy Reverso Apache (.htaccess)

Como o servidor institucional CTI expõe externamente a porta HTTPS padrão (`443`) gerenciada pelo Apache HTTP Server, o arquivo `.htaccess` encaminha de forma transparente

todo o tráfego recebido para o processo local do Node.js/Express (`127.0.0.1:3333` ou porta atribuída `59057`), preservando o cabeçalho `Authorization`:

```
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteBase /26-presco/

    RewriteCond %{HTTP:Authorization} .
    RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]

    <IfModule mod_headers.c>
        RequestHeader set X-Forwarded-Proto https
        RequestHeader set X-Forwarded-Port 443
    </IfModule>

    RewriteRule ^(.*)$ http://127.0.0.1:3333/$1 [P,L]
</IfModule>

Options -Indexes
```

2. Gerenciamento de Processos com PM2 (ecosystem.config.cjs)

O processo da API é mantido ativo continuamente pelo **PM2** em modo `fork` (garantindo captura integral dos streams `stdout` e `stderr` do `tsx`), com reinício automático em caso de falha e teto de memória de `1 GB`:

```
pm2 start /var/www/equipes/26-presco/ecosystem.config.cjs
pm2 save
```

3. Segurança HTTP e CORS em Produção (src/backend/src/app.ts)

- **Headers Defensivos:** Todas as respostas incluem `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection: 0` e `Referrer-Policy: no-referrer`.
- **Política de CORS:** Permite chamadas nativas do aplicativo mobile (que não enviam header `Origin` de navegador) e restringe chamadas web estritamente aos domínios listados em `ALLOWED_ORIGINS`.
- **Execução do Deploy Remoto:** Para disparar o deploy a partir da máquina de desenvolvimento ou via GitHub Actions:

  ```
  npm run deploy
  # Verifique o status e os logs do servidor remoto a qualquer momento
  com:
  npm run remote:status
  npm run remote:logs
  ```

5.4. Procedimento de Build e Distribuição Mobile (APK Android e IPA iOS)

Passo 1: Apontar o Frontend para a API de Produção

Antes de gerar o pacote final do aplicativo móvel, sincronize o ambiente do frontend para apontar para a URL HTTPS de produção:

```
npm run env:prod
```

(Este comando define `EXPO_PUBLIC_API_URL=https://eq.projetoscti.com.br/26-presco` em `src/frontend/.env`).

Passo 2: Gerar o Pacote Android Nativo (.apk)

O projeto suporta duas modalidades de compilação do APK Android:

1. **Build Local Automatizado via Gradle (`scripts/android.ts`):**

   ```
   # Gera o APK de produção assinado sem necessidade de instalar em
   emulador conectado
   npm run android:prod
   ```

   O binário final é gerado em `src/frontend/android/app/build/outputs/apk/release/app-release.apk`.

2. **Build Oficial Autônomo via GitHub Actions (`.github/workflows/release.yml`):** Ao enviar uma tag SemVer (ex.: `git tag v1.5.1 && git push origin v1.5.1`) ou acionar a publicação:

   ```
   npm run release
   ```

   O workflow na nuvem executa `npm test` e `npm run typecheck`, realiza o `expo prebuild`, compila o binário nativo com JDK 17 e Android SDK e publica automaticamente o arquivo `app-release.apk` na página oficial do **GitHub Releases** do projeto.

Passo 3: Geração de Pacote iOS (.ipa)

Para dispositivos Apple (iOS), o workflow `.github/workflows/ios-build.yml` executa em *runners* `macos-latest` com Xcode, gerando o artefato `.ipa` sem assinatura paga para instalação via *Sideloadly* ou *AltStore*.

5.5. Procedimento de Build e Deploy da Landing Page Web (GitHub Pages)

A Landing Page oficial (`src/landing/`) apresenta o sistema, demonstra as capturas de tela reais do aplicativo e exibe o QR Code vetorial ISO/IEC 18004 para download direto do APK.

Passo 1: Compilar o Bundle Estático Web Localmente

```
npm run landing:build
```

O comando executa `expo export -p web --output-dir dist` dentro de `src/landing/`, gerando os arquivos HTML, CSS e JavaScript otimizados no diretório `src/landing/dist/`.

Passo 2: Publicação Multi-Ambiente no GitHub Pages (.github/workflows/deploy-landing.yml)

O deploy da Landing Page ocorre automaticamente a cada push:

- **Branch `main` (Produção):** Publicada na raiz `https://criperrr.github.io/INI3A-EQ3/` com `EXPO_BASE_URL=/INI3A-EQ3`.
- **Branch `dev` (Staging):** Publicada de forma isolada em `https://criperrr.github.io/INI3A-EQ3/dev/`.
- **Branches de Feature / Pull Requests:** Geram prévias isoladas em `https://criperrr.github.io/INI3A-EQ3/preview/<slug>/`.

5.6. Esteira de CI/CD de 4 Camadas (GitHub Actions)

Para garantir estabilidade contínua em produção, o repositório implementa uma arquitetura de integração e entrega contínua dividida em **4 camadas automatizadas** em `.github/workflows/`:

1. **Camada 1 — Gatekeeper de Pull Requests (`pr-gatekeeper.yml`):** Valida nomenclatura de branch, padrão de commits (*Conventional Commits*), tipagem estática (`npm run typecheck`), análise estática de segurança SAST (Semgrep) e suíte de testes unitários em menos de 3 minutos.
2. **Camada 2 — Pipeline de Staging (`staging.yml` e `maestro-e2e.yml`):** Executado após merges na branch `dev`; roda os testes de concorrência (`race_conditions.test.ts`), valida os fluxos E2E mobile no emulador Android com aceleração KVM via **Maestro** e compila o APK de homologação (`Presco-Staging-devFlavor.apk`).
3. **Camada 3 — Promoção e Deploy em Produção (`production.yml` e `deploy.yml`):** Ao promover código para `main`, sincroniza cirurgicamente o backend via SSH com o servidor CTI (`/var/www/equipes/26-presco`), aplica migrações, reinicia o PM2, valida o `/health` público e compila o APK de produção (`prodFlavor`).
4. **Camada 4 — Auditoria Noturna de Performance e Resiliência (`nightly-audit.yml`):** Executada diariamente às 02:00 UTC, submetendo a API aos cenários de carga e estresse do **Grafana k6** e analisando automaticamente logs via `scripts/audit_failure_analyzer.js`.